



Gra łącząca nawet najmniej zgraną grupę znajomych.
Dostępna na tu i teraz, bez zbędnych instalacji.

Architektura aplikacji

Nasz system to architektura rozproszona składająca się z trzech serwisów:

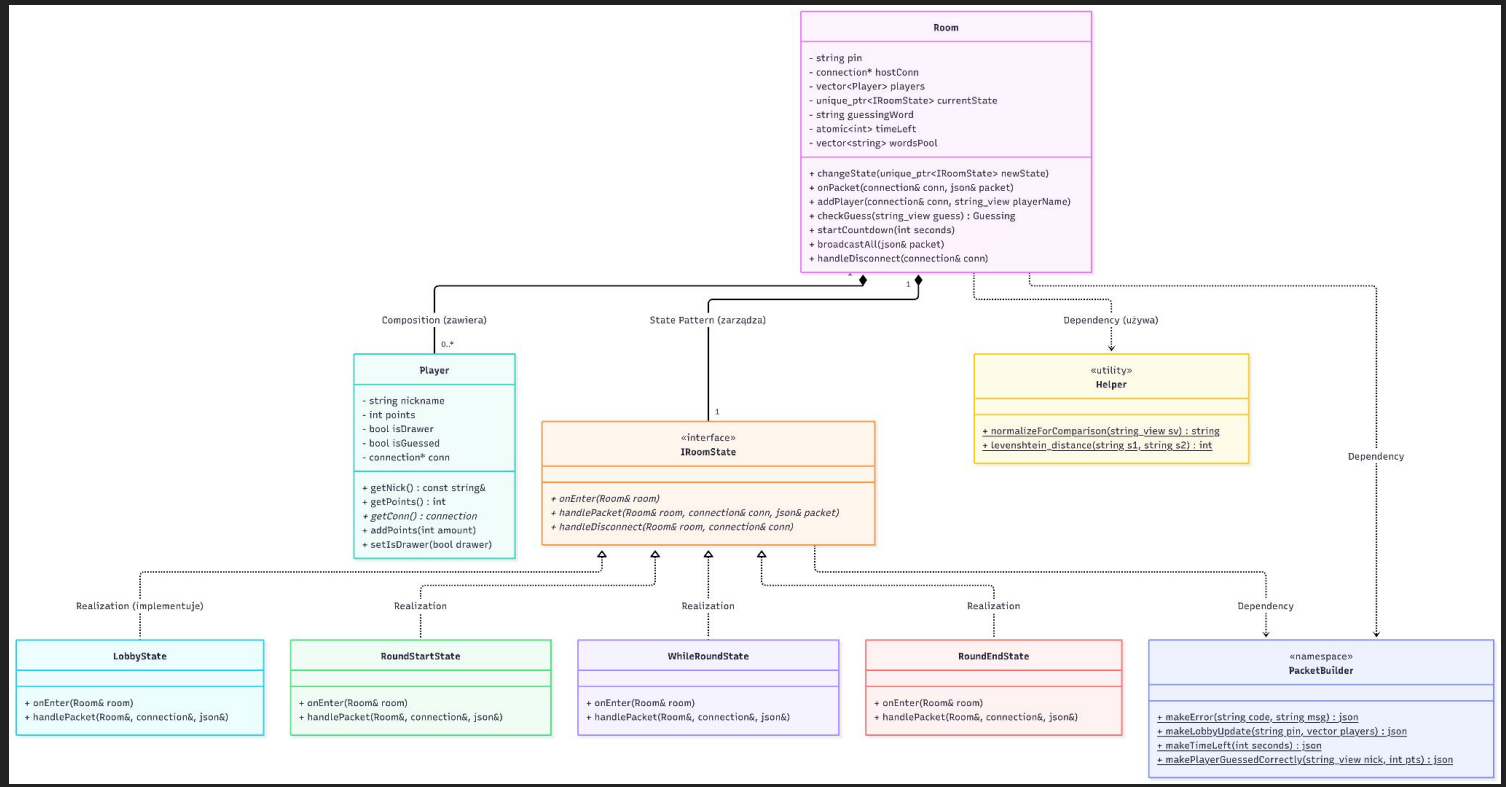
- Frontend (React): Odpowiada za UI Hosta i Gracza.
- Główne API (ASP.NET Core): Autoryzacja/Autentykacja, generowanie kodów PIN, zapis historii gier, praca z bazą danych.
- GameServer (C++ / Crow): Szybki serwer WebSocketowy, zarządzający wyłącznie trwającą rundą w czasie rzeczywistym.

Decyzje Architektoniczne

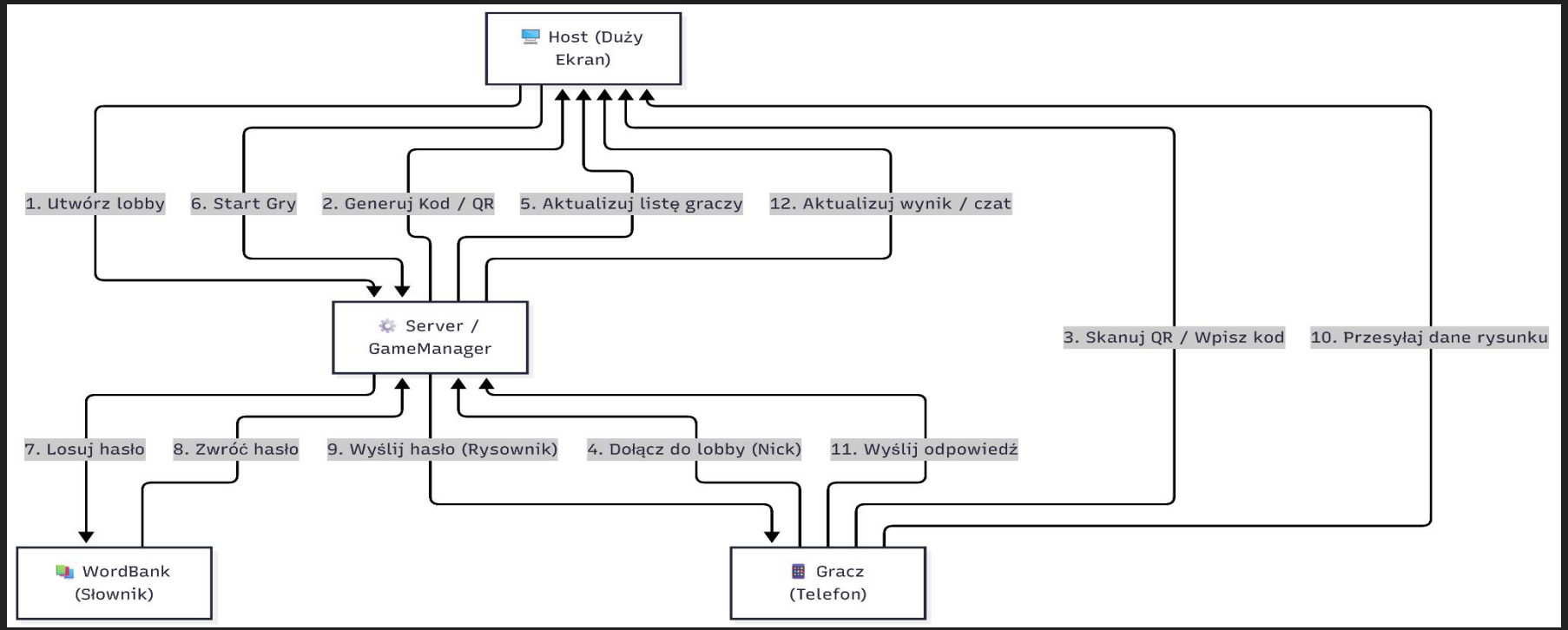
Najważniejsze decyzje z naszego rejestru ADR:

- ADR 0001: Rozdzielenie logiki na ASP.NET (dane statyczne) i C++ (WebSocket) w celu maksymalizacji wydajności.
- ADR 0002: Użycie wzorca State do uniknięcia spaghetti kodu w zarządzaniu fazami gry.
- ADR 0005: Zapis i obsługa historii gier – frontend robi zrzut płótna i wysyła do ASP.NET (odciążenie bazy danych i serwera C++).

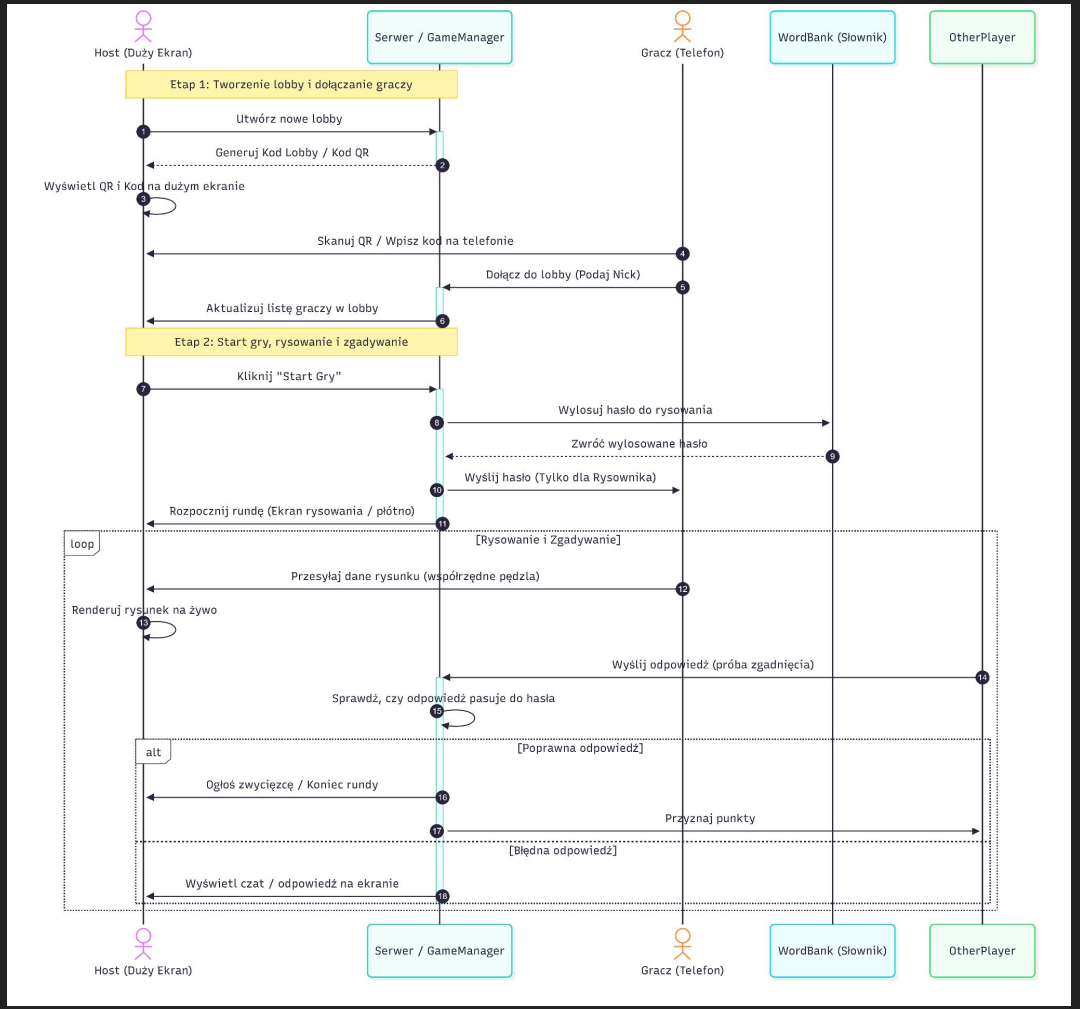
Widok logiczny (UML Class Diagram)



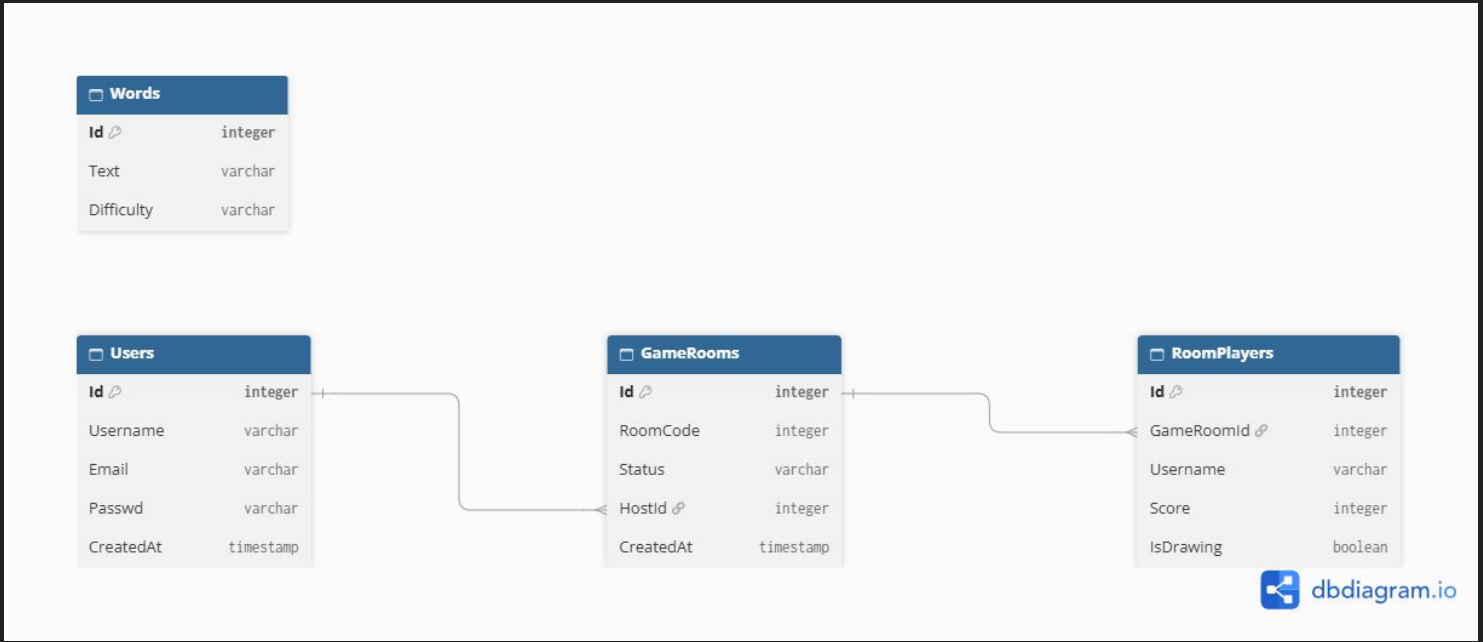
UML Communication Diagram



UML Sequence Diagram

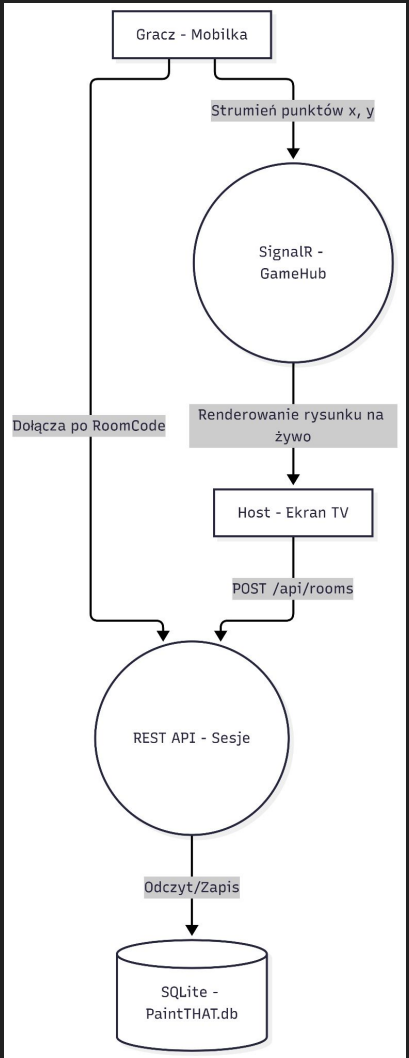


ERD



- Users i GameRooms - relacja 1 do wielu, powiązana kluczem Hostid. Jeden uzytkownik nie moze być hostem wielu rozgrywek
- GameRooms i RoomPlayers - relacja 1 do wielu, powiązana kluczem Gameroomid. Każdy pokój zawiera listę dołączonych graczy, gdzie mamy ich nick i flagę isDrawing, która decyduje kto rysuje
- Words - tabela zawierająca bazę haseł, gdzie do wyboru są poziomy trudności

DFD



- Rest API zbudowane w ASP.NET z bazą w SQLite - cykl życia pokoju i zarządzanie graczami
- SignalR - przekazuje do ekranu hosta render obrazu, który wysyła aplikacja mobilna gracza rysującego
- SQLite PaintTHAT - wszystkie te zapytania REST idą tu, gdzie zapisywany jest stan pokoju

Działająca historia użytkownika

US-04: Szybkie dołączanie

Jako uczestnik rozgrywki, **chcę** móc szybko i łatwo dołączyć do gry, **aby** nie tracić czasu na rejestrację.

- **Given** Host otworzył pokój, a na dużym ekranie widnieje 4-cyfrowy PIN oraz kod QR.
- **When** Gracz zeskanuje kod QR telefonem (lub wpisze PIN) i poda swój Nick.
- **Then** Serwer gry autoryzuje wejście i natychmiast dodaje gracza do poczekalni, bez logowania.

Działająca historia użytkownika cd.

```
std::unordered_map<std::string, std::unique_ptr<Room> >
rooms;

if (type == "PLAYER_JOIN") {
   std::string playerName =
packet.at("payload").at("nickname");
   room.addPlayer(conn, playerName);

   json packetToSend =
Packet::makeLobbyUpdate(room.getPin(),
room.getPlayers());
   room.broadcastAll(packetToSend);

const auto it = rooms.find(PIN);
it->second->onPacket(conn, packet);
```

```
json packet = json::parse(data);

std::string PIN = packet.at("pin");

std::string type = packet.at("type");

std::lock_guard<std::mutex> lock(roomsMutex);

if (type == "HOST_INIT") {

   auto [it, inserted] = rooms.try_emplace(PIN,
std::make_unique<Room>(conn, PIN));

   if (!inserted) {

       conn.send_text(Packet::makeError("ROOM_EXISTS",
"Pokoj o tym PIN-ie juz istnieje").dump());

       return;

   }

   connectionToRoom[&conn] = PIN;
```

Wykorzystanie AI

Nazwa AI	Obszar zastosowania	Rezultaty	% objętości pracy
Gemini	Projektowanie Architektury	Rozdzielenie logiki na ASP.NET i C++. Wybór optymalnego zapisu obrazków	25%
Gemini	C++: Współbieżność i Memory Management	Zabezpieczenie pętli czasu Mutexami, rozwiązanie problemów z gubieniem referencji stringów	40%
Gemini	React: Routing i Refaktoryzacja	Wdrożenie centralnego zarządzania WebSocketem w App.tsx zamiast w poszczególnych widokach, rozwiązanie problemów z asynchronicznością renderowania	35%
Gemini	Generowanie powtarzalnego kodu	Szybsze tworzenie struktur klas, interfejsów TypeScript i formatowania HTML/CSS	40%

